

量子コンピューティングI NISQ編 #3

第7回 中間レポート振り返り + エラー緩和手法

情報工学科 佐藤貴彦

#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12	#13	#14
Intro.	量子回路	Deutsch	Grover	実機実装	QAOA	本日	量子AI	誤り訂正	論理演算	符号理論	量子中継	QIアプリ	総括
基礎編 #2-4 (完)				NISQ 編 #5-8				FTQC 編 #9-11			量子インターネット編 #12-13		

この回の到達点

- ① 中間レポート（ライツアウトxGrover）を振り返る
- ② ゲートエラー緩和技術（ZNE）を実装する
- ③ 測定エラー緩和技術（MEM）を実装する

Section I: 改良版 Grover ライツアウトソルバ

探索空間を大幅に圧縮した上で解決する

解説の流れ

- 1 行依存性の直感 + 形式化
- 2 レジスタ設計と反復構造
- 3 U の中身と inv_k の正体
- 4 Oracle / Diffusion の縮退
- 5 k_{opt} と理論最大成功率
- 6 Qiskit 実装 (関数 5 種)
- 7 動作結果と k 振動の検証
- 8 まとめと発展案

1 行目で残りが一意に決まる仕組みを丁寧に追う (2 枚)

20 qubit、 $U \rightarrow \text{Oracle} \rightarrow U^\dagger \rightarrow \text{Diffusion}$ (2 枚)

5 ステップ概観 + 具体例トレース + flip_3 省略の話 (3 枚)

なぜ第 3 行だけ / 3 qubit だけで済むか (2 枚)

$N=8, M=1 \rightarrow k_{\text{opt}}=2$ 、上限は 94.5% (99% は届かない)

ヘルパー $\rightarrow U \rightarrow \text{Oracle} \rightarrow \text{Diffusion} \rightarrow$ ソルバ統合 (5 枚)

$k=2$ で 94%、 k 振動も理論曲線とほぼ一致 (2 枚)

ナイーブ版との比較サマリ + 中間レポート向け展開

(#4 再掲) ライツアウト

0	1	2
3	4	5
6	7	8

■ 3×3 ライツアウトの特性:

- 解が一意に定まる (5×5 だと複数解 or 解なし)
- 初期配置に関わらず必ず解が存在
- 各ボタンは高々 1 回押せば足りる (冪等性)
- 押す順番は結果に影響しない (可換)

■ 探索空間: $2^9 = 512$ 通り、解は 1 つ ► **M=1, N=512**

- 決定問題なのでオラクル U_f が組める
- Grover で解けるはず!

盤面: lights = [0,1,0,1,1,0,1,1,0] → 1, 3, 4, 6, 7 が点灯

どのボタンを押せば全消灯できる?

全てのボタンは高々 1 回押せば良い
総当たりでも 512 通りで解ける

行依存性に注目する

「ボタン押下の組み合わせが 512 通りある」
と思うと総当りに見えるが、そんなに自由度はない

考えてみよう:

第 1 行の真ん中のタイルだけが点灯している盤面で、
これを消すには？

選択肢:

- ① そのタイル自身のボタン (= タイル 1)
- ② 左隣のボタン (= タイル 0)
- ③ 右隣のボタン (= タイル 2)
- ④ 真下のボタン (= タイル 4)

→ ①～③ は他のタイルも同時に変えてしまう。
「タイル 1 だけを反転できる単独操作」は存在しない。

第 1 行のタイルを消すには、そのタイルの真下のボタン (第 2 行のボタン) を押すしかない。

→ 第 1 行の各タイルが「点灯している/いない」だけを見れば、第 2 行で押すべきボタンが自動で確定する。

同じ論法で:

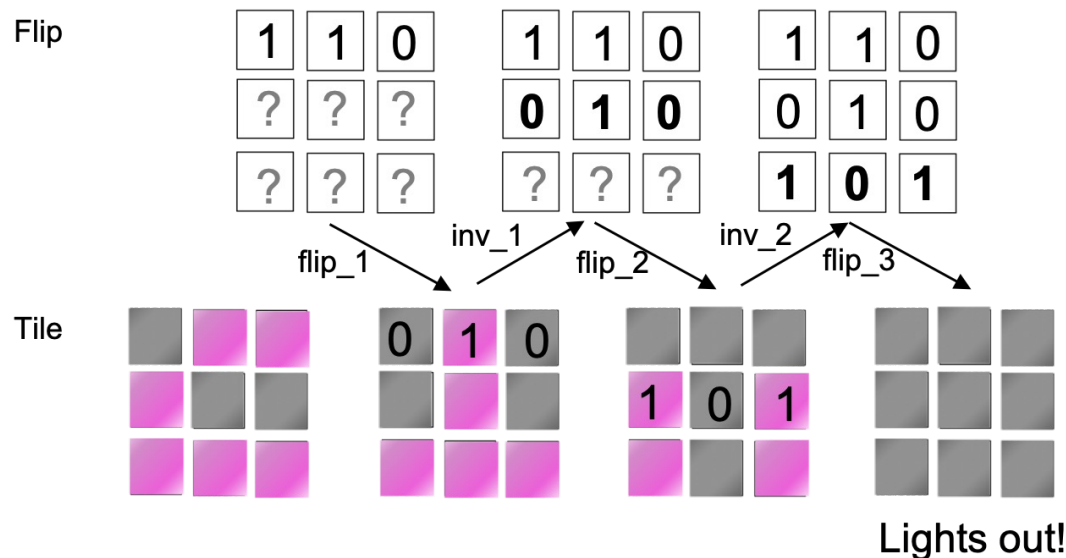
第 2 行の状態 → 第 3 行のボタンが自動確定

つまり「自由な選択」をするのは第 1 行のボタン 3 個だけ。残りは芋づる式 (= chasing the lights)。

探索空間を圧縮する

■ 自由度を数え上げる

- 元の自由度: 9 個のボタンが独立に 0 / 1 $\rightarrow 2^9 = 512$ 通り
- 前スライドの観察から、独立に選べるのは第 1 行の 3 個だけ $\rightarrow 2^3 = 8$ 通り
- 残り 6 個のボタンは 第 1 行の選択 (3 ビット) と初期盤面 lights から決定論的に導かれる
- Grover の探索空間を 64 分の 1 に圧縮 = k_{opt} を激減できる (後のスライドで定量化)



(参考図) 上段の「?」は決まっていないタイルの状態、下段はその時点で押すボタン。第 1 行を決めると右に流れていく

改良版のレジスタ設計 — 20 qubit に収まる

レジスタ構成		
flip	9 qubit	押すボタン (0/1 の 9 個)。H で重ね合わせるのは flip[0:3] だけ
tile	9 qubit	現在の盤面状態 (各タイルの 0/1)。初期化で X を撒いて lights を反映
oracle	1 qubit	位相反転アンシラ。X → H で $ -\rangle$ にしておくのは標準パターン
auxiliary	1 qubit	第 3 行 3-qubit AND の格納先 (mcx の標的)。1 個で足りる
合計 20 qubit (ナীব版 26 qubit に対して 6 qubit 削減)		

ナীব版との対比		
項目	ナীব	改良
合計 qubit	26	20
重ね合わせ qubit (データ空間)	9	3
探索空間 N	$2^9 = 512$	$2^3 = 8$
反復回数 k_opt	17	2
補助量子ビット	7 (Toffoli)	1 (mcx)
P_max (増幅上限)	99.6%	94.5%

ポイント: Diffusion も 3 qubit 上で済む。1 反復あたりのコストも軽くなる

反復構造は変わらない: $U \rightarrow \text{Oracle} \rightarrow U^\dagger \rightarrow \text{Diffusion}$

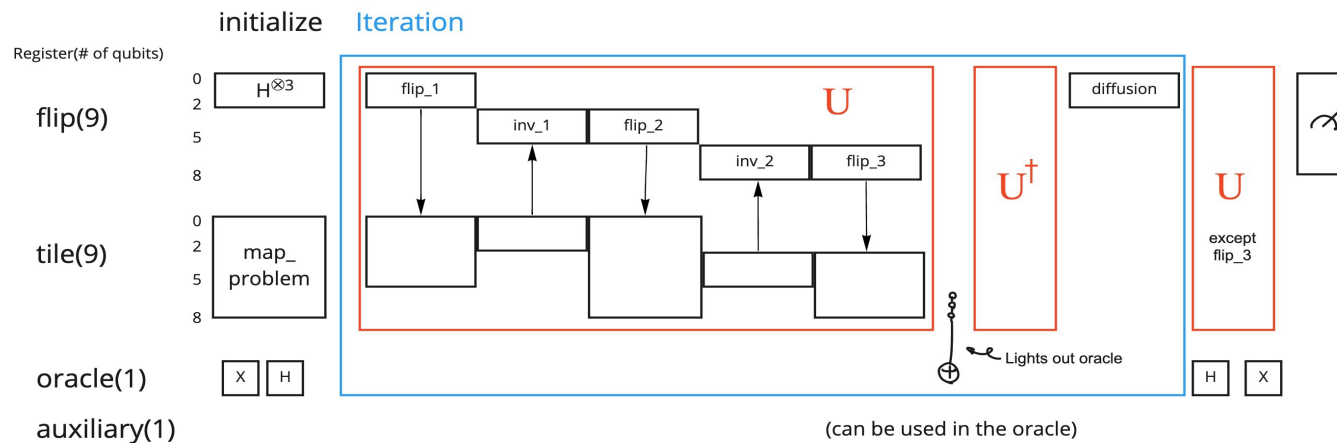
1 回の Grover 反復 (これを $k_{\text{opt}} = 2$ 回繰り返す)

U : 1 行目の選択 flip[0:3] から、flip[3:9] と tile を導出するユニタリ (中身は次スライド)

Oracle : tile[6:9] が全 0 (= 解候補) のとき oracle に位相 -1 をキックバック

$U^\dagger$: U の逆操作。CNOT は自己逆元なのでゲート列を逆順に並べるだけ

Diffusion : flip[0:3] の 3 qubit 空間で標準 Grover Diffusion (H, X, CCZ, X, H)



IBM Quantum Challengeの解説より

(https://github.com/qiskit-community/IBMQuantumChallenge2020/blob/main/solutions/week-2/ex_2a_solutions_ja.ipynb)

反復終了後にもう一度 U を適用 (ただし flip_3 を省く)

→ 増幅された flip[0:3] から flip[3:9] を再構成して、9 ビット全部を測定可能にする (Slide 8 で詳説)

U の中身 — 5 ステップで 盤面を定式化する

flip_1 flip[0:3] を制御に、第 1 行のボタンの隣接タイルへ CNOT = 「第 1 行を実際に押す」

inv_1 tile[0:3] $\rightarrow$ flip[3:6] $\wedge$ CNOT $\times 3$ = 「第 1 行で点灯したままのタイルの真下を押す」

flip_2 flip[3:6] を制御に第 2 行の操作を決め、第 1 行のタイルを全て消灯 (真下のボタンが押される)

inv_2 tile[3:6] $\rightarrow$ flip[6:9] $\wedge$ CNOT 3×3 。第 3 行の操作が flip[0:3] の関数として一意に決まる

flip_3 flip[6:9] で第 3 行を押す。これで第 2 行のタイルも必ず 0 になる

U 実行後の保証 — tile[0:6] は flip[0:3] の選び方によらず必ず 0、tile[6:9] のみが残る

$\rightarrow$ Oracle で 9 タイル全部を見る必要がなく、tile[6:9] の 3 qubit だけ調べれば「3 行目も消えている = 解候補かどうか」が分かる

実装 | ヘルパー関数 — adjacent, flip_row, inv_row

3 つの関数の役割

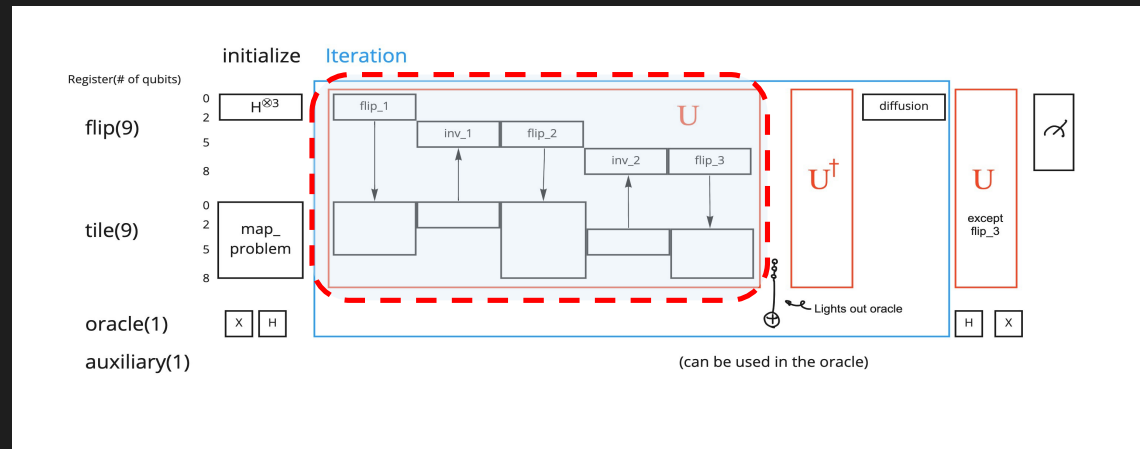
- adjacent(i) : ボタン i を押すと反転するタイル番号のリスト (自身 + 上下左右、盤外は除外)
- flip_row(qc, flip, tile, row) : 第 row 行のボタン 3 個分の flip $\rightarrow$ tile CNOT を一括適用 (= flip_tile の 1 行分)
- inv_row(qc, flip, tile, src_row) : tile[j] $\rightarrow$ flip[j+3] への CNOT 3 本 (chasing rule の量子化)

盤面処理を実装する

```
def adjacent(i):
    """ボタン i を押すと反転するタイル番号 (自身 + 上下左右)"""
    r, c = i // 3, i % 3
    result = [i]
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            result.append(nr * 3 + nc)
    return result

def flip_row(qc, flip, tile, row):
    for i in range(row * 3, row * 3 + 3):
        for t in adjacent(i):
            qc.cx(flip[i], tile[t])

def inv_row(qc, flip, tile, src_row):
    # tile[j] -> flip[j+3] : 第 src_row 行で点灯したタイルの真下を押す
    for j in range(src_row * 3, src_row * 3 + 3):
        qc.cx(tile[j], flip[j + 3])
```



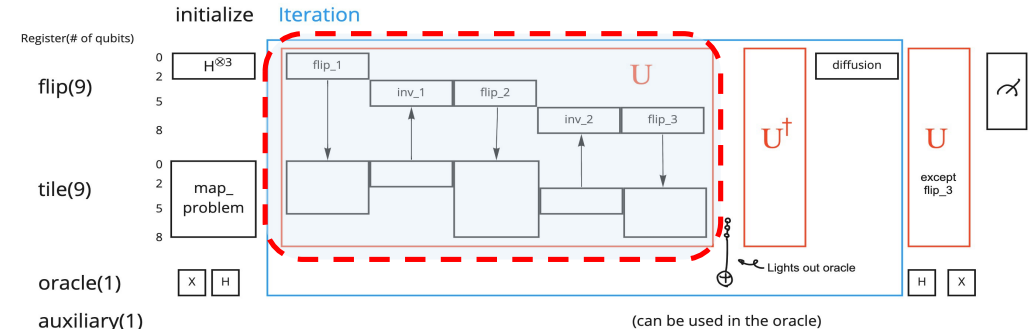
実装 | U_encode と U_uncompute

ポイント

- ・ include_flip_3 オプションで最終 U (測定前準備) のスキップを切り替える
- ・ U_uncompute は U_encode のゲート列を逆順に並べるだけ (qc.inverse() でも可だが、明示的に書くと読みやすい)
- ・ CNOT のみで構成されているため、対 U_encode に対する完全な逆操作になっている (動作確認は notebook で実施済み)

```
def U_encode(qc, flip, tile, include_flip_3=True):
    """flip[0:3] -> flip[3:9] と tile を導出するユニタリ"""
    flip_row(qc, flip, tile, row=0) # flip_1
    inv_row(qc, flip, tile, src_row=0) # inv_1: tile[0:3] -> flip[3:6]
    flip_row(qc, flip, tile, row=1) # flip_2
    inv_row(qc, flip, tile, src_row=1) # inv_2: tile[3:6] -> flip[6:9]
    if include_flip_3:
        flip_row(qc, flip, tile, row=2) # flip_3

def U_uncompute(qc, flip, tile):
    """U_encode の逆。CNOT は自己逆元なのでゲート列を逆順に並べるだけ"""
    flip_row(qc, flip, tile, row=2)
    inv_row(qc, flip, tile, src_row=1)
    flip_row(qc, flip, tile, row=1)
    inv_row(qc, flip, tile, src_row=0)
    flip_row(qc, flip, tile, row=0)
```



具体例で考える: $\text{flip}[0:3] = (0,1,0)$ の場合

初期盤面 $\text{lights} = [0,1,0, 1,1,0, 1,1,0]$ (古典解は ボタン 1, 3, 5, 8 = $\text{flip}[0:3]=(0,1,0)$) flip[0:3] から決定論的に派生する従属変数

ステップ	実行内容 (古典 XOR で追う)	tile [0..8] の状態	flip[3:9] の状態
init	lights を tile に転写	0 1 0 1 1 0 1 1 0	0 0 0 0 0 0
flip_1	flip[0:3]=(0,1,0) で第 1 行を押下	1 0 1 1 0 0 1 1 0	0 0 0 0 0 0
inv_1	tile[0:3] → flip[3:6] にコピー	1 0 1 1 0 0 1 1 0	1 0 1 0 0 0
flip_2	flip[3:6]=(1,0,1) で第 2 行を押下	0 0 0 0 0 1 0 1 1	1 0 1 0 0 0
inv_2	tile[3:6] → flip[6:9] にコピー	0 0 0 0 0 1 0 1 1	1 0 1 0 0 1
flip_3	flip[6:9]=(0,0,1) で第 3 行を押下	0 0 0 0 0 0 0 0 0	1 0 1 0 0 1

ポイント

- flip_1 直後の第 1 行 (1,0,1) がそのまま inv_1 で flip[3:6] に転写され、続く flip_2 で第 1 行が消える
- 最終的に tile が全 0 になったので、flip[0:3]=(0,1,0) が正解。Oracle はこれを検知して位相を反転する

Oracle — 第 3 行 3 タイルだけの AND 検査

■ なぜ 3 タイルで済むのか

- U 実行直後の tile 状態: tile[0:6] は flip[0:3] の値に関わらず必ず 0 (前スライドの保証)
- 残る非自明な情報は tile[6:9] の 3 ビットだけ — これが全部 0 なら flip[0:3] は解
- → Oracle は 3 qubit の「全 0 検査」さえできれば十分 (ナイーブ版の 9 qubit AND は不要)

実装パターン (5 段階)

- | | |
|-----------------------|--|
| ① $X \times 3$ | tile[6], tile[7], tile[8] を反転 — 「全 0 検査」を「全 1 検査」に書き換える |
| ② mcx (3 制御 → aux[0]) | 3 個全部が 1 のときだけ aux[0] が 1 になる (= 3 qubit AND) |
| ③ CX (aux → oracle) | aux[0]=1 のとき oracle qubit ($ -\rangle$ 状態) に位相 -1 をキックバック |
| ④ mcx (再度同じ) | aux[0] を Uncompute — もつれ解消のために必須 |
| ⑤ $X \times 3$ (再度同じ) | tile[6], tile[7], tile[8] を元に戻す |

→ Uncompute (④) を忘れると aux が flip[0:3] ともつれたままになり、Grover が破綻する (#3 で扱った原則)。ここを丁寧に実装することが重要: aux が共有 1 qubit のため、もつれが残ると次反復で間違った位相が波及する

実装 | improved_oracle, 3 qubit AND と位相キックバック

実装のコツ

- ・ mcx は qiskit の汎用多重制御 X。controls に渡すリストの長さを 3 にしておく
- ・ 位相キックバックは「aux=1 のとき oracle (|−⟩) に −1」を実現するために CX(aux, oracle) を入れる
- ・ ④ の Uncompute mcx を抜くと、aux が flip 状態とエンタングルしたまま反復に持ち込まれ、Grover が機能しなくなる

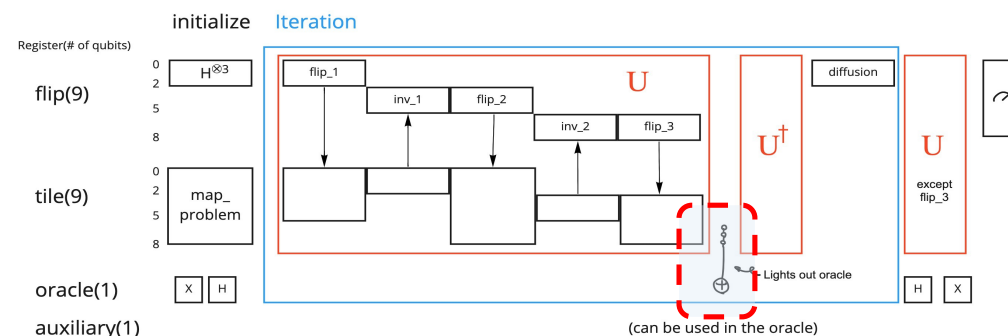
```
def improved_oracle(qc, tile, oracle, aux):
    """tile[6:9] (第 3 行) が全 0 のとき oracle に位相 -1 をキックバック"""
    # ① X で「全 0 検査」を「全 1 検査」に書き換える
    qc.x(tile[6]); qc.x(tile[7]); qc.x(tile[8])

    # ② 3-qubit AND -> aux[0]
    qc.mcx([tile[6], tile[7], tile[8]], aux[0])

    # ③ 位相キックバック (oracle は |−> 状態)
    qc.cx(aux[0], oracle[0])

    # ④ aux[0] の Uncompute (もつれ解消) - ここを忘れると Grover が壊れる
    qc.mcx([tile[6], tile[7], tile[8]], aux[0])

    # ⑤ X を元に戻す
    qc.x(tile[6]); qc.x(tile[7]); qc.x(tile[8])
```



Diffusion: 3 qubit 空間の標準形

■ なぜ flip[0:3] だけでよいのか

- Grover Diffusion の役目は「重ね合わせ qubit の平均についての反転」($a' = 2\mu - a$)
- 改良版では重ね合わせを作っているのは flip[0:3] だけ — flip[3:9] は U の中で決定論的に派生
- つまり「振幅増幅のステージ」も 3 qubit で完結する。3 qubit Diffusion は CCZ 1 個で構成可能

3 qubit Diffusion の構造 (標準パターン)

$$H^{\otimes 3} \rightarrow X^{\otimes 3} \rightarrow CCZ \rightarrow X^{\otimes 3} \rightarrow H^{\otimes 3}$$

- $CCZ = H - CCX - H$ で実装可能 (qiskit には ccz が無いが、target を H で挟むだけ)
- 全体に $\exp(i\pi) = -1$ のグローバル位相を載せる (global_phase += np.pi)
Diffusion を $2|s\rangle\langle s| - I$ の形式 (パターン A) に揃える

ナイーブ版との比較

- 9 qubit Diffusion は $H^{\otimes 9} \rightarrow X^{\otimes 9} \rightarrow 8 \text{ 制御 } Z \rightarrow X^{\otimes 9} \rightarrow H^{\otimes 9}$: 多重制御 Z は 1 段で数十ゲートに分解される
- 改良版 3 qubit Diffusion は CCZ 1 個で済む — 1 反復あたりの Diffusion 部分の深度は 10 分の 1 以下

improved_diffusion — CCZ を 1 個書くだけ

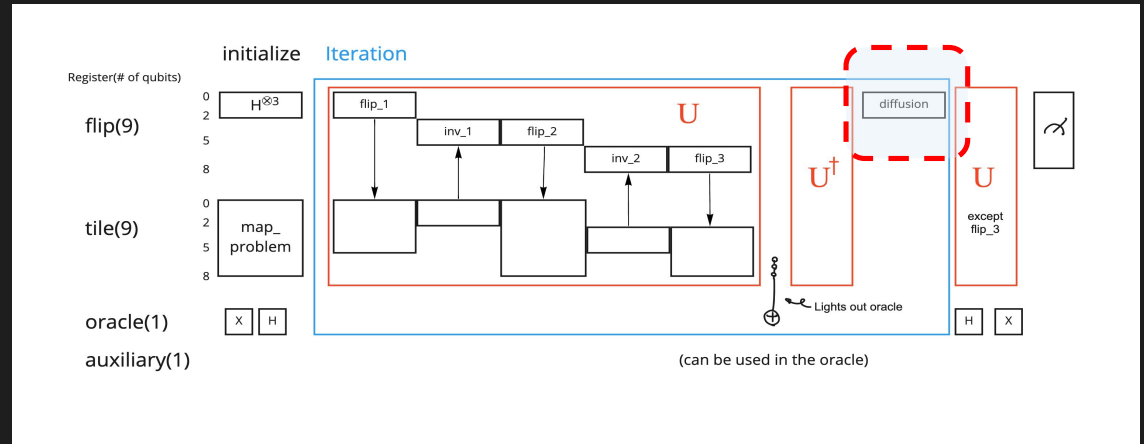
実装のコツ

- qiskit に CCZ ゲートは存在しないので、 $H - CCX - H$ で挟む (target 側だけ H)
- $\text{global_phase} += \text{np.pi}$ はナイーブ版 Diffusion と統一するためのパターン A 化 — 数式の符号合わせに必要なだが計算結果には影響しない
- $\text{flip}[0:3]$ 以外の qubit には一切作用しない — H や X を $\text{flip}[3:9]$ かけるとバグる

```
def improved_diffusion(qc, flip):
    """flip[0:3] の 3 qubit 上で標準 Grover Diffusion"""
    qc.h([flip[0], flip[1], flip[2]])
    qc.x([flip[0], flip[1], flip[2]])

    # CCZ = H - CCX - H (target qubit を H で挟む)
    qc.h(flip[2])
    qc.ccx(flip[0], flip[1], flip[2])
    qc.h(flip[2])

    qc.x([flip[0], flip[1], flip[2]])
    qc.h([flip[0], flip[1], flip[2]])
    qc.global_phase += np.pi # パターン A 化
```



繰り返し回数 k_{opt} と理論最大成功率 = 94.5%

公式 (#3 復習)

$$\theta = \arcsin \sqrt{M / N}$$

$$k_{\text{opt}} = \lfloor \pi / (4 \theta) \rfloor$$

$$P_{\text{success}}(k) = \sin^2((2k+1) \theta)$$

(N : 探索空間, M : 解の個数)

改良版Groverの具体値

$$N = 2^3 = 8, M = 1 \text{ (解は一意)}$$

$$\theta \approx 0.3614 \text{ rad}$$

$$k_{\text{opt}} = \lfloor \pi / (4 \times 0.3614) \rfloor = \lfloor 2.174 \rfloor = 2$$

$$P_{\text{max}} = \sin^2(5\theta) \approx 0.9453 \text{ (= 94.5\%)}$$

ポイント

- $N=8, M=1$ の整数反復 Grover では $\sin^2(5\theta) \approx 0.9453$ が構造的上限
- ちょうど $(2k+1)\theta = \pi/2$ の連続最適値 $k^* \approx 1.674$ と整数 $k=2$ にはズレがある
- 「99% 以上」などを旨すには Fixed-point Grover や Oblivious Amplitude Amplification 等の追加技法が必要
- 94.5% であれば解が他状態と桁違いに突出するため、観測 1 回でもほぼ正解が引けるはず

反復後の「except flip_3」が意味すること

■ 反復終了時点で何が起きているか

- Grover 反復後、Diffusion は flip[0:3] の振幅だけを増幅している
- U[†] で flip[3:9] は |000000> に戻されているため、そのまま測定しても 0 ばかりになる
- 9 ビットの完全な解を取り出すには、flip[0:3] から flip[3:9] を再構成する必要がある

データ構造を復元する必要がある

対処: 反復ループの後にもう一度 U を適用する (flip_3 だけ省く)

- ・ flip_1 → inv_1 → flip_2 → inv_2 までで flip[3:9] は確定 (chasing rule で芋づる)
- ・ flip_3 は tile を最終消灯させるだけ — flip 値の決定には寄与しない → スキップ可
- ・ 測定するのは flip レジスタだけ。tile は捨てるので 0 にする必要がない

コスト計算と代替案

- ・ 余分なゲート数: U_encode (flip_3 抜き) 1 回分のみ ≒ Grover 反復 1 回の 1/3 程度
- ・ k_opt = 2 の改良版にとっては 1/6 強の上乗せ。それでもナイーブ版より圧倒的に軽い
- ・ 別解: 反復後すぐに flip[0:3] だけ測定し、古典で flip[3:9] を計算する手もある

build_improved_grover — 統合ソルバ

構造: 初期化 (flip[0:3] のみ H、tile に lights、oracle に $|-\rangle$) $\rightarrow$ 反復 $k_{\text{opt}}=2 \rightarrow$ 測定前準備 $U_{\text{encode}}(\text{include_flip_3}=\text{False})$

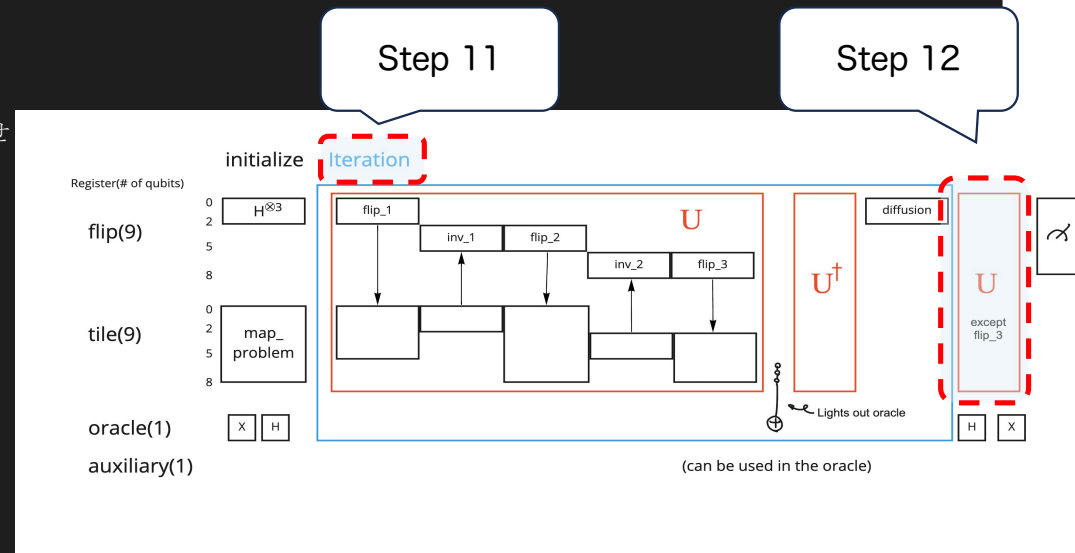
```
def build_improved_grover(lights, num_iterations):
    flip = QuantumRegister(9, 'flip'); tile = QuantumRegister(9, 'tile')
    oracle = QuantumRegister(1, 'oracle'); aux = QuantumRegister(1, 'aux')
    cr = ClassicalRegister(9, 'c')
    qc = QuantumCircuit(flip, tile, oracle, aux, cr)

    # 初期化
    qc.h([flip[0], flip[1], flip[2]])
    for i, l in enumerate(lights):
        if l == 1: qc.x(tile[i])
    qc.x(oracle[0]); qc.h(oracle[0])

    # 第 1 行のみ重ね合わせ
    # 盤面をエンコード
    # oracle を  $|-\rangle$  に

    #  $k_{\text{opt}}$  反復
    for _ in range(num_iterations):
        U_encode(qc, flip, tile)
        improved_oracle(qc, tile, oracle, aux)
        U_uncompute(qc, flip, tile)
        improved_diffusion(qc, flip)

    # 測定前: flip[3:9] を flip[0:3] から再構成 (flip_3 は省略)
    U_encode(qc, flip, tile, include_flip_3=False)
    qc.measure(flip, cr); return qc
```



k = 2 で測定する: 94% で解が得られる

実行条件: lights = [0,1,0, 1,1,0, 1,1,0], shots = 2048, AerSimulator (理想系), 実行時間 約 1 秒
古典解: ボタン 1, 3, 5, 8 を押す → qiskit 表記 (little-endian) で 100101010

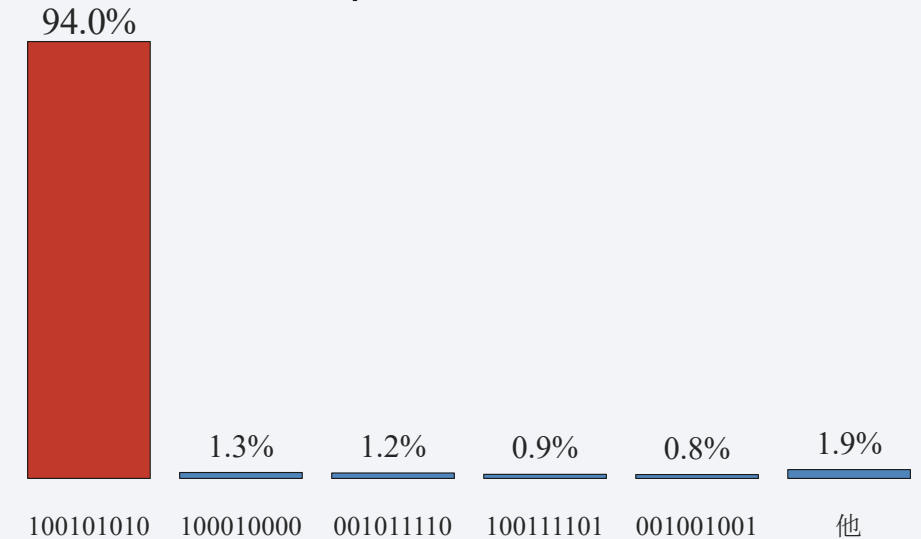
Top 5 観測

100101010	1924	(93.95%) ← 解
100010000	26	(1.27%)
001011110	24	(1.17%)
100111101	18	(0.88%)
001001001	17	(0.83%)

→ 解が 2 位以下と 50 倍以上乖離する

観測 1 回でも高確率で正解が分かる

確率分布の概略 (Top 6)



考察: k と 理論曲線

k = 0..6 の成功率 (1024 shots / 盤面固定)

k	実験	理論 $\sin^2((2k+1)\theta)$
0	0.146	0.125
1	0.776	0.781
2	0.938 ← 最適	0.945
3	0.340	0.330
4	0.006	0.012
5	0.579	0.548
6	1.000	≈ 1.000

→ 実験値が理論値とほぼ一致。k=2 が確かに最適、k=6 で次の周期ピーク

複数盤面での検証 (k = 2 固定 / 1024 shots)

盤面 lights	P(解)
[0,1,0,1,1,0,1,1,0]	0.938
[1,0,1,0,1,0,1,0,1]	0.942
[1,1,1,1,1,1,1,1,1]	0.958
[1,0,0,0,0,0,0,0,0]	0.953
[0,0,0,0,0,0,0,0,0] (全消灯)	0.946

→ どの盤面でも約 94% で解を観測 = 理論上限通り。盤面非依存に動作

理論との整合性

実験値 (AerSimulator) と理論曲線 $P(k) = \sin^2((2k+1)\theta)$ のずれは shots 由来の統計揺らぎ程度。回路設計に誤りがないことが定量的に確認できる

Sec. I まとめ

■ 改良版で変わったこと

- 行依存性により 重ね合わせ qubit を $9 \rightarrow 3$ に圧縮 (探索空間 $512 \rightarrow 8$ 、64 分の 1)
- Oracle の検査範囲も $9 \rightarrow 3$ タイル、auxiliary qubit は $7 \rightarrow 1$ (大型Toffoli)
- 反復回数 $17 \rightarrow 2$ 、実行時間は AerSimulator で 12 秒 $\rightarrow$ 1 秒未満
- 理論最大成功率は $99.6\% \rightarrow 94.5\%$ にやや低下 ($N=8$, $M=1$ の整数反復 Grover の構造的上限)

■ さらなる改良の余地は・・・？

- 列依存性 / 対称性 (例: D4 対称) を加味すれば探索空間をさらに削減
- Fixed-point Grover や Quantum Amplitude Amplification 改良版で 94.5% の壁を超える
- Oracle 内部の AND 構造を ZX-calculus などですらに浅くする
- QAOAで用いた線型方程式系: $aX = b$ をオラクル化する などさまざまなGroverの性能を引き出す余地がある

Section II: エラー緩和技術

エラー訂正ができないNISQの性能を引き出す手法

QEC: Quantum Error Correction

将来のFTQC = 量子誤り訂正

- 実行 中 に誤りを検出・訂正
- 大量の物理量子ビット (~1000:1) とシンドローム測定が必要
- 原理的に任意の誤りを訂正可能
- FTQC (Fault-Tolerant) 時代の技術

QEM: Quantum Error Mitigation

現代のNISQ = 量子誤り緩和

- 実行 後 に古典後処理で誤りを補正
- サンプル数を増やしてバイアスを除去
- 完全な訂正は不可能
サンプル数オーバーヘッドと引き換え
- NISQ 時代の現実的解 ← 本講義

NISQで顕在化する4種のエラー

エラーは回路のどこで何が引き起こしているか

デコヒーレンス

T1 (緩和), T2 (位相緩和)

時間と共に量子状態が古典的な状態へ移行する
回路を深くするほど影響が大きい

クロストーク

近接量子ビット間の意図しない結合

あるqubit操作が隣のqubitに影響を与える
本講義では深入りしない

ゲートエラー

1量子ゲート $\sim 10^{-4}$, 2量子ゲート $\sim 10^{-2}$

ユニタリ操作が完全に実行されない
パルス制御の精度限界に起因

ZNE で緩和

測定エラー

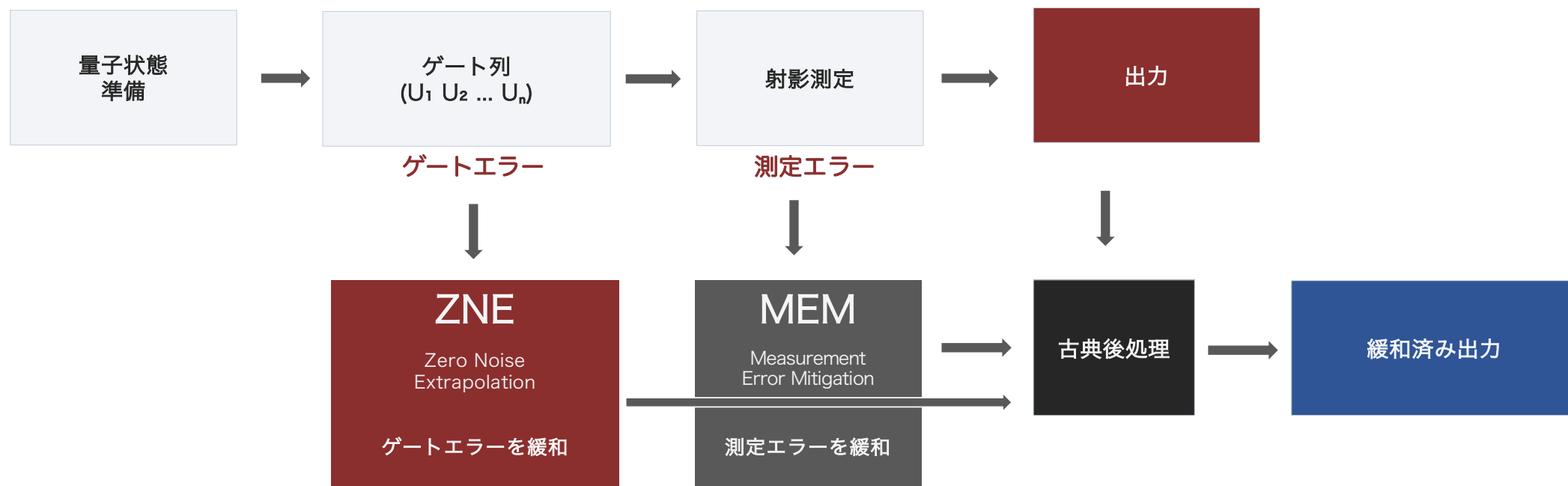
読み出しエラー $\sim 1-3\%$

本当は $|0\rangle$ なのに $|1\rangle$ と読み出される (逆もある)
比較的大きなエラー源

MEM で緩和

本日の2手法と位置づけ

ゲート と 測定 の独立した処理



両者は独立な物理現象に対応するため別個に設計され、組み合わせて使うこともできる

Part I: 測定エラーを軽減したい

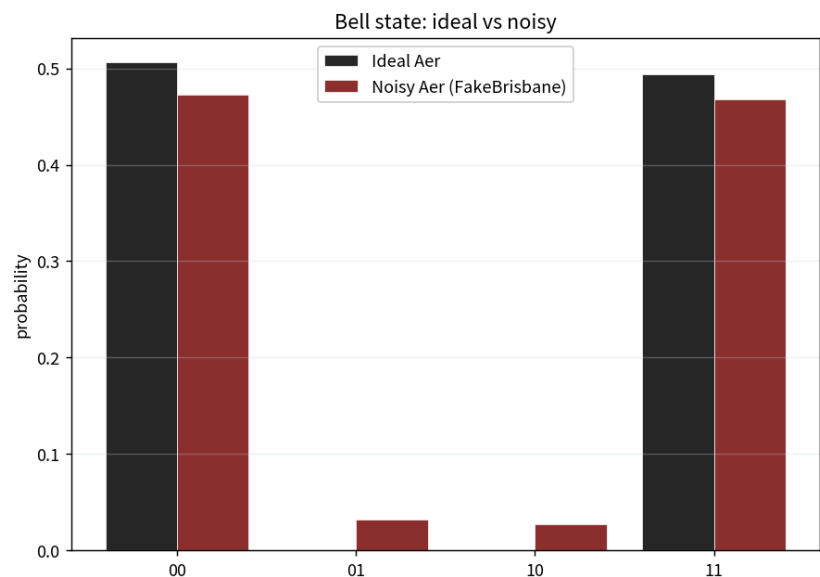
ベル状態を実機で作ると何が起きるか

例: ibm_brisbane でベル状態を作って測定
($|00\rangle + |11\rangle$)/ $\sqrt{2}$ を 1024 shots 測定

測定結果 (過去の講義データより)

$|00\rangle$: 509
 $|01\rangle$: 42
 $|10\rangle$: 40
 $|11\rangle$: 433

→ 理想なら $|00\rangle=|11\rangle\approx 512$, $|01\rangle=|10\rangle=0$ のはず



MEM: 読み出しエラー (Measurement Error) を軽減 (Mitigation) する

→ 「測定エラー」だけを特定して取り除く方法はあるか？それが MEM のテーマ

重要: 測定は線形写像である

測定エラーに逆問題からアプローチする

$$\mathbf{p_meas} = \mathbf{A} \cdot \mathbf{p_true}$$

観測される確率分布 = 遷移行列 × 真の確率分布

順問題 (forward)

「真値」が分かっているば
測定でどう乱れるかは予測できる

$$A_{\{ji\}} = \Pr(\text{出力}=j \mid \text{真値}=i)$$

対角優位 (diagonal-dominant):
ほとんどは正しく読まれる

具体的にどうやるのかは次ページ

逆問題 (inverse) ← MEM はこれ

観測値から真値を推定:

$$\mathbf{p_true} \approx \mathbf{A}^{-1} \cdot \mathbf{p_meas}$$

- $\mathbf{A}$ を事前に知る必要がある
- $\mathbf{A}^{-1}$ を測定結果に作用させる

注意: 結果が確率分布である保証なし

MEM の実装: サンプルングで遷移行列を作る

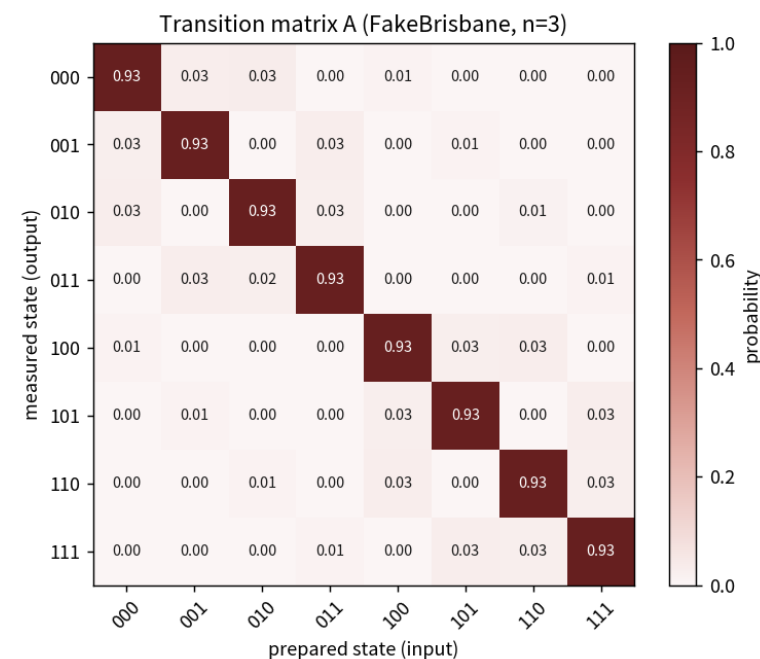
全基底状態を準備して測定する

測定結果の校正手順

- 1 全 2^N 個の基底状態 $|i\rangle$ を準備
- 2 それぞれを即測定
- 3 観測カウントを正規化して A の i 列目に格納
- 4 実回路の測定結果に A^{-1} を作用

[注意点] コスト: 校正回路は 2^N 個
 $N=10 \rightarrow 1024$ 回, $N=20 \rightarrow 100$ 万回

スケールしない手法
であることに留意する



FakeBrisbane の $N=3$ 遷移行列 (対角優位)

逆行列適用の落とし穴

負の確率と非物理的な解が出現することもある

$A^{-1} \cdot p_{\text{meas}}$ は「確率分布である保証がない」

→ 負の確率や、和が1にならない結果が普通に出る

対処法

1. クリッピング

$\max(p, 0)$ して再正規化

+ シンプルで実装容易

– 情報を失う、誤差にバイアス

2. NNLS (制約付き)

$\min \|Ax - p\|^2 \quad \text{s.t. } x \geq 0$

+ 理論的に正しい

– 凸最適化が必要、遅い

3. Quasi-probability

負も許容したまま期待値計算

+ 期待値計算には正確

– 確率分布の意味は失う

次ページで取り扱うM3 では quasi-probability をデフォルトとし、必要に応じて `nearest_probability_distribution()` で物理化

M3 (Mthree): スケーラブルな測定誤り緩和

M3 = Matrix-free Measurement Mitigation: IBM Quantum 発の Qiskit 公式アドオン mthree

なぜナイーブな方法は使えないか

N	校正回路数	行列サイズ
3	8	8×8
10	1,024	$1,024 \times 1,024$
20	1,048,576	$10^6 \times 10^6$ (非現実的)

スケールする手法が
ライブラリ化されている！

M3 のアイデア

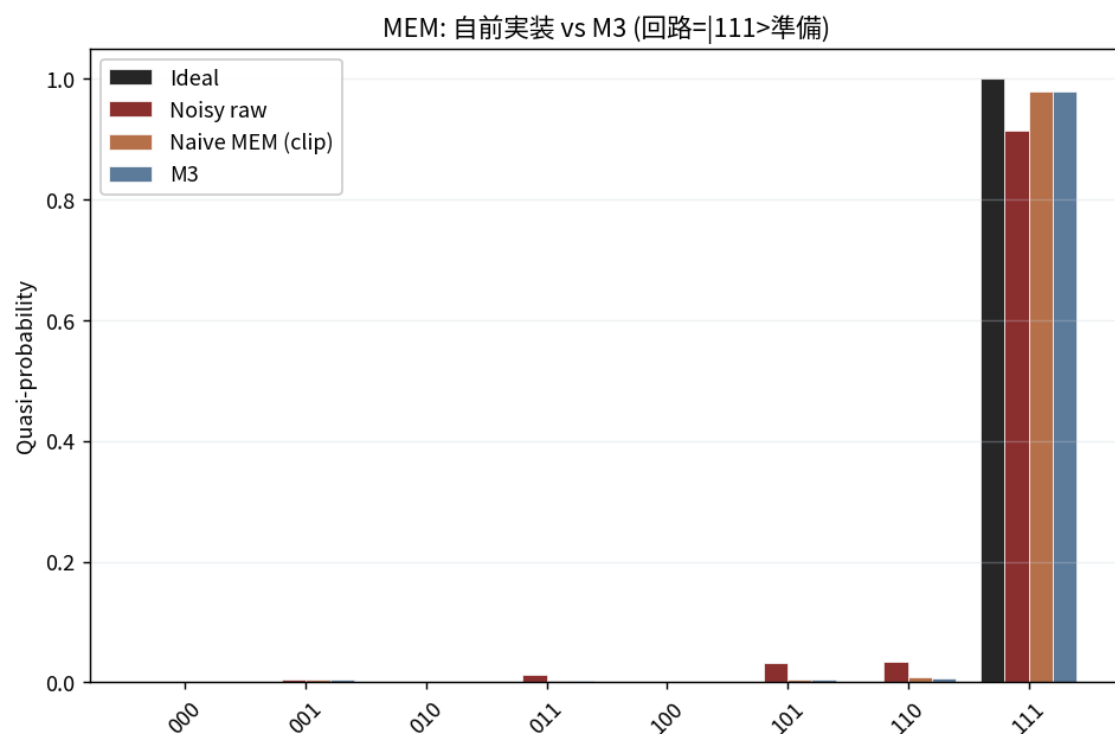
- ① テンソル積構造を活用
量子ビット毎の校正のみ
→ 2^N ではなく $2N$ 個の校正
- ② 観測ビット列のみで部分空間
4096回測定で 30種類しか出なければ
30×30 の問題に縮約

結果: 42 qubit でも数秒で補正可能

```
mit = mthree.M3Mitigation(backend); mit.cals_from_system(qubits); quasi = mit.apply_correction(counts, qubits)
```

結果: 自前実装 vs M3

$|111\rangle$ 準備回路 (X-CX-CX) で実験する



$P(|111\rangle)$ と理想値からの距離

方法	$P(111\rangle)$	L1 距離
Ideal	1.000	0.000
Noisy raw	0.913	0.175
Naive MEM	0.978	0.044
M3	0.981	0.039

注意: $N=3$ では両者の差は小さい

M3 の真価は $N \geq 10$ で現れる:
 ナイーブな手法では計算不能、M3は数秒

Part II: ゲートエラーを軽減したい

ZNE (Zero Noise Extrapolation) ノイズを故意に増幅 + 外挿でゼロにする..ことを目指す

$$\langle 0 \rangle(\lambda) = \langle 0 \rangle_{\text{ideal}} + c_1 \lambda + c_2 \lambda^2 + \dots$$

ノイズの強さ λ で観測期待値を展開, $\lambda \rightarrow 0$ で理想値

① 測定

$\lambda = 1, 3, 5, \dots$ で実行

② フィット

観測値に多項式や指数を当てる

③ 外挿

$\lambda \rightarrow 0$ の極限を求める

ZNE の強み: ハードウェアに直接アクセスできないユーザでも実装可能

ZNE の前提: ノイズが回路深さに「ある程度」比例して増える (脱分極ノイズが典型)

ZNE が苦手なもの: SPAM (State Preparation and Measurement) エラー => 外挿では消せない

ノイズを増幅する主要な方法

ゲートフォールディングが教科書的

Pulse stretching

パルス長を伸ばす

- + 物理的に最も正確
- パルス制御権限が必要

Global folding

回路全体に $U^\dagger U$ を挿入

- + 実装が簡単
- ハード境界での揺らぎ

Local folding

各ゲート G を $G G^\dagger G$ に置換
(gate-wise)

- + きめ細かい制御
- + 本講義のデフォルト

本講義

理想: $G G^\dagger = I \rightarrow G G^\dagger G = G$ (論理動作は不変, ノイズだけ3倍)

実装の注意点

フォールディングは `transpile level = 0` で行おう

素の回路に余剰なゲートを追加 → `transpile` すると余剰なゲートが消えてしまう

× WRONG

素の回路 qc
→ `fold_locally(qc)`
→ `transpile(opt=1)`

**transpiler (qiskitのコンパイラ)が G G+ を I に
畳んでしまい増幅効果が消失**

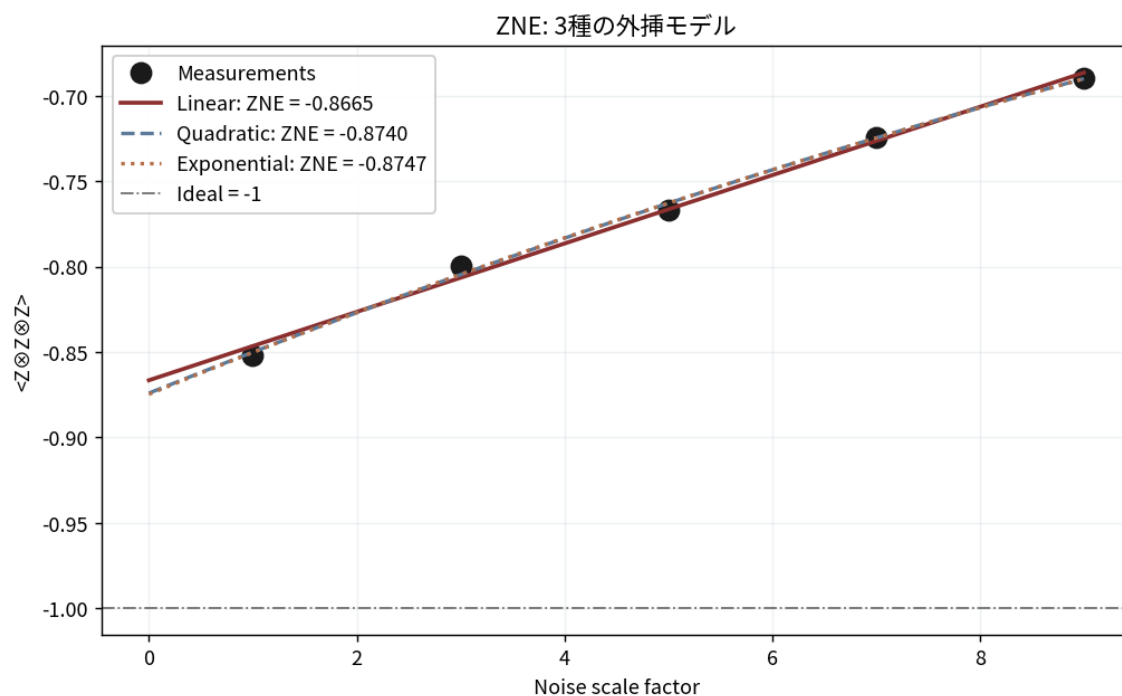
○ CORRECT

素の回路 qc
→ `pm1.run(qc)` [opt=1, ISA化]
→ `fold_locally(isa)`
→ `pm0.run(folded)` [opt=0, 再展開]

再 `transpile` は最適化を切る (level 0)

補足 | 外挿モデルの選択

暗黙のノイズモデルが何かを意識する



3つの外挿モデル

線形

$$a \cdot \lambda + b$$

1次近似 (小ノイズ極限)

2次多項式

$$a \cdot \lambda^2 + b \cdot \lambda + c$$

1次補正で足りない場合

指数

$$A \cdot e^{-B \lambda} + C$$

脱分極ノイズの典型形

実機・深い回路では指数モデルが優位。外挿点が少ない / ノイズが弱い領域では線形で十分

ZNE の限界と前提

効果的にNISQの出力を改善するが、決して万能ではない



SPAMエラーは緩和不能

状態準備・測定におけるエラーは「ゲート」ではないので対処できない
MEMで別途処理する必要がある

MEMと組み合わせる場合は
実行順も重要になる（後述）



サンプリング数の増加

scale_factor 1 つにつき shots × 2-3倍 必要となる
M個の scale で M倍になり、バイアスを減らす代わりに分散が増える

多くのNISQアルゴリズムは
出力が確率分布なので分散は大敵



外挿モデル依存のバイアス

「正しい外挿モデル」が分らないと、モデル選択自体が新しい誤差源になる
実機で fitting 残差を確認必要がある

長大な回路の場合、適切な
外挿モデルを構成できないことも

ZNE は バイアス除去 と サンプル数増加 をトレードオフしている

MEM × ZNE を併用する場合

順序が大事: MEM が先, ZNE が後

観測される期待値:

$$\langle O \rangle_{\text{obs}}(\lambda) = f(\lambda) \cdot \langle O \rangle_{\text{true}} + \delta_{\text{SPAM}}$$

$f(\lambda)$: ゲートノイズ由来 (λ で外挿可)

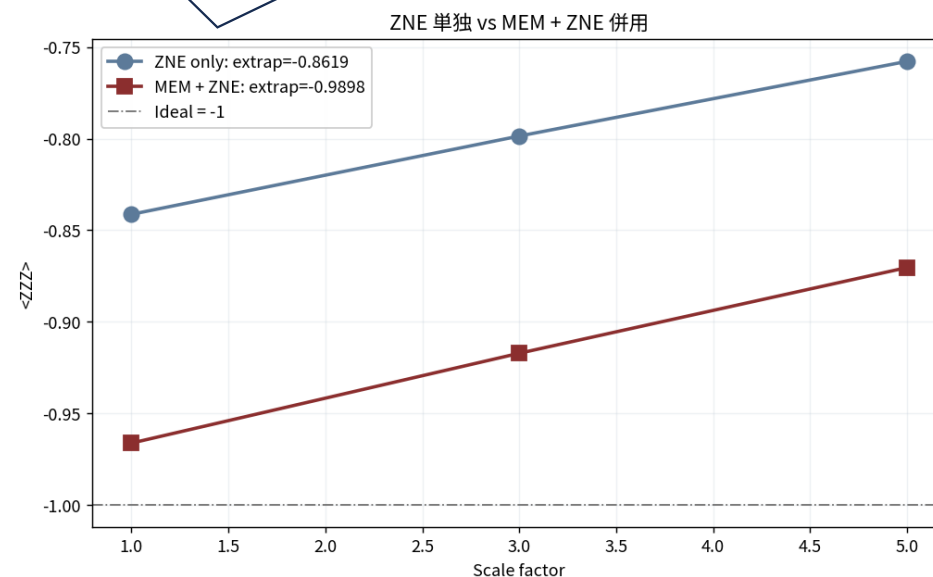
δ_{SPAM} : 測定エラー由来 (加法バイアス)

先に MEM で δ_{SPAM} を除去

後で残った $f(\lambda) \cdot \langle O \rangle_{\text{true}}$ を ZNE で外挿

逆順だと δ_{SPAM} ごと外挿してしまう結果となる

SEED値で揺らぐことに注意
(研究・卒論では誤差も書く)



ほぼノイズレスな
出力結果が得られている

実験結果 (理想値 -1.0):

Noisy raw: -0.83 / ZNE only: -0.85

MEM + ZNE: -0.99 ← ほぼ理想値

QEM技術の関連研究

他の主要なQEM手法

PEC

Probabilistic Error Cancellation

ノイズチャネルを擬確率分布で逆転。最も理論的に厳密だがオーバーヘッドが大きい

CDR

Clifford Data Regression

Clifford 近似回路で機械学習的にノイズ補正を学習

対称性検証

Symmetry Verification

ハミルトニアンの対称性を後測定で検証, 違反するショットを破棄

参考文献

Endo, Benjamin, Li

PRX 2018

Practical QEM for Near-Future Apps (サーベイ)

Temme, Bravyi, Gambetta

PRL 2017

Error Mitigation for Short-Depth Circuits (ZNE)

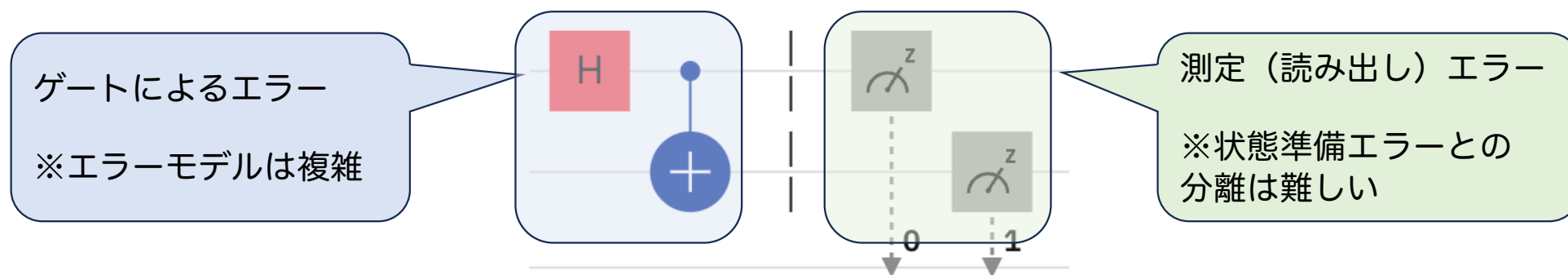
Nation et al.

PRX Quantum 2021

Scalable Mitigation of Measurement Errors (M3)

Sec. II まとめ

- 量子ビットの寿命 (T_1, T_2) 以外にもさまざまなエラーがある
 - ✓ 例：ゲートエラー、測定エラー



- エラーの影響をmitigation（緩和・軽減）するさまざまな手法が提案されており、本講義では以下の2手法を紹介した
 - Measurement Error mitigation
 - 測定エラーの緩和
 - 遷移行列を用いる手法が一般的
 - Zero Noise Extrapolation
 - ゲートエラーの緩和
 - エラーを故意に増幅し、エラーがない状態を逆算する

量子コンピューティングIA 最終課題

選択自由

1. 5Qubit以上を用いたアルゴリズムを**fake backend**で実行し、各自が設定した問題を解決しなさい
 - ✓ 誤り緩和手法を組み合わせ、有効性を確認しなさい（緩和無しの結果と比較すること）
 - 本講義で触れていないアルゴリズム・手法を用いてもよい
 - 複数の手法を組み合わせてもよい
 - 外部からソースコードを引用した場合には明示すること
 2. [緊急避難用] NISQアルゴリズムを2つ調査しなさい
- 問題設定・実行内容・結果・考察などを1~3ページのレポートにまとめなさい
 - ✓ 1を選択した場合はnotebookなどの実装コードを添付してください

締め切り 6/10 (水) 23:59